

Quantum Channel Language Model (QCLM): Architecture, Mathematics, and Implementation

QCLM Project

June 5, 2026

Contents

1 Overview	2
2 Mathematical Formulation	2
2.1 State space: the density matrix	2
2.2 Token operators: Kraus maps and POVM elements	2
2.3 The isometry constraint	3
2.4 The autoregressive law (Born rule + collapse)	3
2.5 Joint probability and the quantum forward algorithm	3
2.6 The role of quantum coherence	4
3 Parametrization and Constraint Enforcement	4
3.1 Free parameters	4
3.2 Differentiable polar projection (isometry enforcement)	4
4 Training	4
4.1 Loss: negative log-likelihood in nats and bits	4
4.2 Efficiency: training cost is independent of vocabulary size	5
4.3 Truncated backpropagation through time (TBPTT)	5
5 The Decoherence Ablation	5
6 Computational Structure and Engineering	6
6.1 Per-token state update	6
6.2 Real 2×2 block matmuls for tensor-core utilization	6
6.3 Compiled chunked scan	6
7 Architecture Summary	7
8 Scaling	7
8.1 Parameter count	7
8.2 Vocabulary is computationally cheap	7
8.3 Scaling the Hilbert-space dimension	7
9 Connection to Prior Work	8

1 Overview

The **Quantum Channel Language Model (QCLM)** is an autoregressive generative model whose sequential context-tracking mechanism is neither self-attention (as in Transformers) nor a gated nonlinear recurrence (as in RNNs, LSTMs, or GRUs), but a *quantum channel* acting on a density matrix in a finite-dimensional Hilbert space.

At each time step the model holds its entire context as a single $n \times n$ complex density matrix ρ_t . Reading a token x_t is a *measurement*: the next-token probability is given by the Born rule $p(x_t | x_{<t}) = \text{Tr}(\rho_{t-1} M_{x_t})$. Observing the token *collapses and evolves* the state through a completely-positive, trace-preserving (CPTP) map — a quantum channel. There is no attention, no gate, and no nonlinear activation. The only nonlinearity in the entire model is quantum measurement itself.

One-line identity. The QCLM is a *complex-valued, trace-preserving linear state-space model with a Born-rule (quadratic) readout* — the quantum cousin of modern linear SSMs (S4, Mamba), distinguished by genuine amplitude **interference**.

2 Mathematical Formulation

2.1 State space: the density matrix

Let $\mathcal{H} = \mathbb{C}^n$ be a Hilbert space of dimension n (the *Hilbert-space dimension*, a hyperparameter). The model carries a *quantum state*

$$\rho \in \mathbb{C}^{n \times n}, \quad \rho = \rho^\dagger, \quad \rho \succeq 0, \quad \text{Tr}(\rho) = 1,$$

called a *density matrix* (or density operator). A *pure state* takes the form $\rho = |\psi\rangle\langle\psi|$ for some unit vector $|\psi\rangle \in \mathbb{C}^n$; a *mixed state* is a convex combination of pure states and encodes classical uncertainty over quantum superpositions. The model begins each sequence in a learned pure state $\rho_0 = |\psi_0\rangle\langle\psi_0|$.

2.2 Token operators: Kraus maps and POVM elements

Each vocabulary token $x \in \{1, \dots, V\}$ owns W complex $n \times n$ matrices called **Kraus operators**:

$$\{K_{x,w}\}_{w=1}^W, \quad K_{x,w} \in \mathbb{C}^{n \times n}.$$

They define two objects simultaneously:

1. **Quantum channel** (completely-positive map):

$$E_x(\rho) = \sum_{w=1}^W K_{x,w} \rho K_{x,w}^\dagger.$$

2. **POVM element** (positive operator):

$$M_x = \sum_{w=1}^W K_{x,w}^\dagger K_{x,w} \succeq 0.$$

2.3 The isometry constraint

A single global constraint ties all tokens together:

$$\sum_{x=1}^V \sum_{w=1}^W K_{x,w}^\dagger K_{x,w} = I_n. \quad (1)$$

Equation (1) is equivalent to $\sum_x M_x = I_n$, which makes $\{M_x\}_{x=1}^V$ a valid *positive operator-valued measure* (POVM). This single equation is what guarantees that next-token probabilities are exactly normalized — by construction, with no softmax.

Proposition 1 (Exact normalization). *Under constraint (1), for any density matrix ρ ,*

$$\sum_{x=1}^V \text{Tr}(\rho M_x) = \text{Tr}\left(\rho \sum_x M_x\right) = \text{Tr}(\rho I_n) = \text{Tr}(\rho) = 1.$$

2.4 The autoregressive law (Born rule + collapse)

Generation and likelihood scoring follow the standard postulates of quantum measurement:

$$\text{Predict: } p(x_t \mid x_{<t}) = \text{Tr}(\rho_{t-1} M_{x_t}) \quad (2)$$

$$\text{Update: } \rho_t = \frac{E_{x_t}(\rho_{t-1})}{p(x_t \mid x_{<t})} \quad (3)$$

Equation (2) is the **Born rule**: the probability of outcome x_t equals the expectation of the corresponding POVM element. Equation (3) is the **post-measurement state update**: apply the quantum channel E_{x_t} and renormalize. Note that

$$\text{Tr}(E_{x_t}(\rho_{t-1})) = \text{Tr}(\rho_{t-1} M_{x_t}) = p(x_t \mid x_{<t}),$$

so the probability is just the trace of the (unnormalized) post-update state — it falls out for free, with no extra computation.

2.5 Joint probability and the quantum forward algorithm

Unrolling the recurrence, the joint probability of a string $x_1, \dots, x_T$ is

$$p(x_1, \dots, x_T) = \text{Tr}(E_{x_T} \circ \dots \circ E_{x_1}(\rho_0)) = \text{Tr}(\tilde{\rho}_T), \quad (4)$$

where $\tilde{\rho}_T = E_{x_T}(\dots(E_{x_1}(\rho_0))\dots)$ is the *unnormalized* final state. Equivalently, the total negative log-likelihood telescopes:

$$\mathcal{L} = -\sum_{t=1}^T \log p(x_t \mid x_{<t}) = -\log \text{Tr}(\tilde{\rho}_T), \quad (5)$$

because each per-step log-probability is $\log \text{Tr}(\tilde{\rho}_t) - \log \text{Tr}(\tilde{\rho}_{t-1})$, and the sum telescopes to $\log \text{Tr}(\rho_0) - \log \text{Tr}(\tilde{\rho}_T) = -\log \text{Tr}(\tilde{\rho}_T)$. This is trained directly by gradient descent (the *quantum forward algorithm*).

2.6 The role of quantum coherence

Between measurements the density matrix carries *off-diagonal* entries ρ_{ij} with $i \neq j$. These encode complex relative phases between basis states. When the Born rule computes the next probability,

$$p(x) = \text{Tr}(\rho M_x) = \sum_{i,j} \rho_{ij} (M_x)_{ji},$$

these off-diagonal terms contribute through *quantum interference*: different “paths” (Kraus operator sequences) can add or subtract in amplitude before squaring. A classical model has $\rho_{ij} = 0$ for $i \neq j$ by construction, so it cannot exploit this interference.

3 Parametrization and Constraint Enforcement

3.1 Free parameters

The Kraus operators $\{K_{x,w}\}$ are the model’s trainable parameters. We store them as separate real and imaginary parts:

$$\mathbf{Kr} \in \mathbb{R}^{V \times W \times n \times n}, \quad \mathbf{Ki} \in \mathbb{R}^{V \times W \times n \times n},$$

so that $K_{x,w} = \mathbf{Kr}[x, w] + i \cdot \mathbf{Ki}[x, w]$. The initial pure state is parametrized as a free complex vector $|\psi_0\rangle = \mathbf{psi0_r} + i \cdot \mathbf{psi0_i}$, normalized on each forward pass.

Total trainable parameters: $2 V W n^2 + 2n$.

3.2 Differentiable polar projection (isometry enforcement)

The raw parameters do not in general satisfy the isometry constraint (1). We enforce it *on every forward pass* via a differentiable polar projection.

Stack all $V \cdot W$ Kraus operators row-wise:

$$\mathbf{V} = \begin{bmatrix} K_{1,1} \\ K_{1,2} \\ \vdots \\ K_{V,W} \end{bmatrix} \in \mathbb{C}^{(VWn) \times n}.$$

The Gram matrix $G = \mathbf{V}^\dagger \mathbf{V} \in \mathbb{C}^{n \times n}$ satisfies $G = \sum_{x,w} K_{x,w}^\dagger K_{x,w}$. We project:

$$\mathbf{V}_{\text{iso}} = \mathbf{V} G^{-1/2}, \tag{6}$$

where $G^{-1/2}$ is computed via Hermitian eigendecomposition $G = U \Lambda U^\dagger \Rightarrow G^{-1/2} = U \Lambda^{-1/2} U^\dagger$, clamping small eigenvalues for numerical stability. By construction, $\mathbf{V}_{\text{iso}}^\dagger \mathbf{V}_{\text{iso}} = I_n$, so constraint (1) holds exactly.

The map $\mathbf{V} \mapsto \mathbf{V}_{\text{iso}}$ is differentiable (via `torch.linalg.eigh`), so gradients flow to the unconstrained parameters through the projection.

4 Training

4.1 Loss: negative log-likelihood in nats and bits

The training objective is the sequence **negative log-likelihood** (NLL) averaged over tokens:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p(x_t | x_{<t}) \quad (\text{nats/token}). \tag{7}$$

This is the *quantum forward algorithm*: at each step, the probability $p(x_t | x_{<t}) = \text{Tr}(E_{x_t}(\rho_{t-1}))$ is read from the trace of the channel output, and the state is updated by normalizing that output.

Bits per token. The equivalent metric in bits is

$$\text{bits/token} = \frac{\mathcal{L}}{\ln 2} = \log_2 \text{PPL}, \quad (8)$$

where $\text{PPL} = e^{\mathcal{L}}$ is perplexity. Bits per token is the information-theoretic cross-entropy: it measures how many bits the model needs on average to encode each token under its predicted distribution. The theoretical minimum is the true entropy of the data (unknown in practice); lower is better. The relation is simply a change of logarithm base and carries no additional information beyond the NLL.

4.2 Efficiency: training cost is independent of vocabulary size

A key property follows from the isometry constraint. The Born probability $p(x_t \mid x_{<t}) = \text{Tr}(E_{x_t}(\rho_{t-1}))$ is the trace of the channel output *for the observed token only*. Computing this requires

$$\tilde{\rho}_t = \sum_{w=1}^W K_{x_t, w} \rho_{t-1} K_{x_t, w}^\dagger,$$

which involves W matrix multiplications of size $n \times n$. Because $\text{Tr}(\tilde{\rho}_t)$ is the probability and the state update is just $\rho_t = \tilde{\rho}_t / \text{Tr}(\tilde{\rho}_t)$, there is **no need to evaluate the full vocabulary sum** during training. The per-token training cost is $O(Wn^3)$, independent of V .

This stands in contrast to transformer language models, where the final softmax over the vocabulary is $O(Vd)$ per token. For a vocabulary of $V = 16,384$ and Hilbert dimension $n = 128$, the QCLM training cost is $O(4 \cdot 128^3) \approx 8\text{M}$ operations per token, regardless of vocabulary size.

4.3 Truncated backpropagation through time (TBPTT)

The quantum state ρ_t depends on all previous tokens $x_1, \dots, x_t$ through the sequential recurrence. For long sequences, retaining the full computation graph is memory-intensive. We apply **truncated BPTT**: every K steps the quantum state is detached from the computation graph,

$$\rho_{tK} \leftarrow \text{stop_gradient}(\rho_{tK}),$$

so gradients are propagated through at most K steps at a time. This bounds activation memory without changing the forward recurrence (the state evolution is unchanged; only the gradient path is truncated).

5 The Decoherence Ablation

To isolate the contribution of quantum coherence, we define a **decohered** (classical) twin: after every state update, we zero the off-diagonal entries of ρ ,

$$\rho_t^{\text{cl}} = \text{diag}(\rho_t^{\text{diag}}) / \text{Tr}(\text{diag}(\rho_t^{\text{diag}})), \quad (9)$$

where ρ_t^{diag} is the vector of diagonal entries of ρ_t .

This operation:

1. **Exactly** reduces the QCLM to a classical n -state *Hidden Markov Model* (HMM). The diagonal $(\rho_{11}, \dots, \rho_{nn})$ plays the role of a classical probability vector over n hidden states.
2. **Preserves** all parameters (V, W, n , Kraus operators). The decohered and quantum models have identical parameter counts and sizes.
3. Provides a **clean, dimension-matched ablation**: any performance difference between quantum and decohered models must come from the off-diagonal coherences, i.e., from quantum interference — not from parameter count or architectural differences.

6 Computational Structure and Engineering

6.1 Per-token state update

The hot path in `forward()` performs, at each step t :

1. **Channel application:** $\tilde{\rho}_t = \sum_w K_{x_t,w} \rho_{t-1} K_{x_t,w}^\dagger$
Implemented as two batched matrix multiplications: $\tilde{K}_w = K_{x_t,w} \rho_{t-1}$, then $E = \sum_w \tilde{K}_w K_{x_t,w}^\dagger$.
2. **Trace:** $p_t = \text{Tr}(\tilde{\rho}_t) = \sum_i (\tilde{\rho}_t)_{ii}$ (Born probability).
3. **Normalize:** $\rho_t = \tilde{\rho}_t / p_t$.
4. **NLL accumulation:** $\mathcal{L} -= \log p_t$.

6.2 Real 2×2 block matmuls for tensor-core utilization

Modern NVIDIA GPUs (Ampere, Hopper, Blackwell) have tensor cores that accelerate real `bfloat16/float16` matrix multiplications but do not natively accelerate `complex64`. We implement each complex $(n \times n) @ (n \times n)$ product as a single real $(2n \times 2n) @ (2n \times n)$ product in `bfloat16`:

$$\begin{bmatrix} C_r \\ C_i \end{bmatrix} = \underbrace{\begin{bmatrix} A_r & -A_i \\ A_i & A_r \end{bmatrix}}_{A\text{-block, } (2n \times 2n)} \underbrace{\begin{bmatrix} B_r \\ B_i \end{bmatrix}}_{B\text{-block, } (2n \times n)}, \quad (10)$$

where $A = A_r + iA_i$ and $B = B_r + iB_i$ are the complex factors. This replaces four separate real multiplications with one larger call, which gives a single kernel launch and a larger tile for the tensor cores to saturate. The `fp32` master weights in `Kr`, `Ki` are unaffected; only the matmul compute is in `bfloat16`. The isometry projection stays in `float32` and `complex64` for numerical stability (called once per forward pass, not per token).

6.3 Compiled chunked scan

The Python-loop bottleneck. The default forward pass runs a Python `for t in range(L)` loop over all L tokens. Each iteration launches several GPU kernels and incurs Python interpreter overhead ($\sim 10 \mu\text{s}$ /iteration), amounting to $\sim 5 \text{ ms}$ for $L = 512$ — significant for fast GPUs.

Chunked compiled scan. We replace the L -iteration Python loop with:

1. **Pre-gather:** compute `Kr_seq = Kr_iso[tokens]` of shape $(B \times L \times W \times n \times n)$ in a *single* batched index operation, instead of L per-token gather kernels.
2. **Outer loop** over $\lceil L/K \rceil$ TBPTT chunks (e.g., 4 iterations for $L = 512$, $K = 128$). At each chunk boundary the state is detached.
3. **Compiled inner scan:** each chunk body runs through `torch.compile(fullgraph=True, dynamic=False)`, which (a) unrolls the inner `for t in range(K)` loop at trace time, (b) fuses the block matmuls, trace, log, and divide across all K iterations into a small set of Triton kernels, and (c) eliminates Python interpreter overhead within the chunk entirely.

The Python interpreter overhead drops from $\sim L$ to $\sim L/K$ iterations; the GPU sees a fused kernel stream rather than L separate launches.

Why not a parallel Blleloch scan? The composition $E_{x_t} \circ \dots \circ E_{x_1}$ is associative, so a parallel prefix scan exists in principle. However, its elements are Liouville transfer matrices of size $n^2 \times n^2$: for $n = 128$, each matrix is $16,384 \times 16,384$ complex $\approx 4 \text{ GB}$. This is memory-infeasible for any practically useful n . The chunked compiled scan is the practical optimum.

7 Architecture Summary

Table 1 collects the key quantities.

Table 1: QCLM architecture at a glance.

Quantity	Expression	Notes
State	$\rho \in \mathbb{C}^{n \times n}$	density matrix, Hermitian, PSD, $\text{Tr } \rho = 1$
Token operator	$\{K_{x,w}\}_{w=1}^W \subset \mathbb{C}^{n \times n}$	W Kraus ops per token
Channel	$E_x(\rho) = \sum_w K_{x,w} \rho K_{x,w}^\dagger$	CPTP map
POVM element	$M_x = \sum_w K_{x,w}^\dagger K_{x,w}$	PSD
Constraint	$\sum_{x,w} K_{x,w}^\dagger K_{x,w} = I_n$	normalization
Prediction	$p(x_t x_{<t}) = \text{Tr}(\rho_{t-1} M_{x_t})$	Born rule
State update	$\rho_t = E_{x_t}(\rho_{t-1}) / p(x_t x_{<t})$	collapse + renorm
Loss	$\mathcal{L} = -\frac{1}{T} \sum_t \log \text{Tr}(E_{x_t}(\rho_{t-1}))$	nats/token
Bits/token	$\mathcal{L} / \ln 2$	$= \log_2 \text{PPL}$
Parameters	$2VWn^2 + 2n$	real numbers; complex = 2 reals
Per-token cost	$O(Wn^3)$	<i>independent of V</i>

8 Scaling

8.1 Parameter count

The dominant cost is the Kraus parameter table:

$$\text{params} \approx 2VWn^2,$$

where the factor 2 counts real and imaginary parts. Parameters live in the per-token Kraus operators, analogously to a joint embedding + unembedding table. For the 2B target configuration:

Config	V	n	W	Params
PoC (char)	65	48	4	$\approx 1.2\text{M}$
1B (BPE-8k)	8,192	128	4	$\approx 1.07\text{B}$
2B (BPE-16k)	16,384	128	4	$\approx 2.15\text{B}$

8.2 Vocabulary is computationally cheap

Because training cost is $O(Wn^3)$ per token (independent of V), increasing the vocabulary from characters ($V = 65$) to BPE subwords ($V \sim 10^4$) multiplies the parameter count by $V_{\text{new}}/V_{\text{old}}$ but does *not* increase training FLOPs. The practical binding constraints are memory bandwidth (loading the Kraus table) and the $O(n^3)$ dense linear algebra per step.

8.3 Scaling the Hilbert-space dimension

Increasing n increases both model capacity ($\propto n^2$ in params) and per-token compute ($\propto n^3$). PoC experiments confirm that the quantum advantage over the decohered HMM baseline *grows* with n :

n	Quantum (bits/char)	Classical (bits/char)	Advantage Δ
24	3.008	3.208	0.200
48	2.806	3.045	0.239

9 Connection to Prior Work

Hidden Quantum Markov Models (HQMMs). The QCLM is a generative HQMM [1, 2]. Prior work trained HQMMs on low-order toy sequences using EM. The QCLM differs in (i) end-to-end gradient descent with a clean differentiable isometry parametrization, (ii) application to natural language at scale, and (iii) the controlled decoherence ablation that isolates interference.

Linear state-space models. S4 [3], Mamba [4], and related models use a linear recurrence $h_t = Ah_{t-1} + Bu_t$. The QCLM’s recurrence $\rho_t = E_{x_t}(\rho_{t-1})/p(x_t)$ is also linear in ρ_{t-1} (for fixed x_t), but the state is a *density matrix* (complex, Hermitian, PSD) and the readout is the Born rule (quadratic in amplitudes) rather than a linear projection. The key distinction is the capacity for *genuine quantum interference* through off-diagonal coherences.

Quantum machine learning. Parameterized quantum circuits (PQCs) are typically studied in the context of near-term quantum hardware [5]. The QCLM is a classical simulation of a quantum process, using quantum *mathematics* (not hardware) as the computational substrate, running efficiently on CPUs and GPUs.

References

- [1] A. Monras, A. Beige, and K. Wiesner. Hidden quantum Markov models and non-adaptive read-out of many-body states. *Applied Mathematics & Computational Science*, 1:93–107, 2010.
- [2] S. Srinivasan, G. Gordon, and B. Boots. Learning hidden quantum Markov models. In *Proceedings of AISTATS*, 2018.
- [3] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *Proceedings of ICLR*, 2022.
- [4] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.
- [5] M. Cerezo et al. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.